CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 06/Unit 06: Constructors, Overloading and Destructors

Course taught by: Zubair Irshad

Courtesy of some slides: Faisal Alvi

Unit Outline

1. Review of Last Week's Content: Classes and Objects
2. A deeper look at constructors
3. Constructors: Initializer Lists
4. Overloading Constructors
5. Destructors
6. Summary.

Classes and Objects: Recap

- Recall from your last session on classes and objects.
- A class in Object Oriented Programming is a collection of
  - **data**, and
  - the **operations** on that data.
- In terms of C++ programming, a class consists of variables that may be primitive data types or classes, as well as functions (on these variables).
- In order to initialize a class, you need to make a **constructors**

Constructors: Recap

- A constructor is a member function that is executed automatically whenever an object is created.
  - Constructors have the same name as the class.
  - A constructor has no return type associated with it.
- Earlier we saw that a constructor can be initialized using
  - either an initializer list,
  - or by assigning values to instance variables in the body of the constructor.
- **Question: When are initializer list important**?

```
MyClass() : member1(val1), member2(val2) {
    // constructor body (optional)
}
```


initializer list

Overloading Constructors

How to initialize objects with a variety of values?

Overloading Constructors

- Constructors may sometimes be overloaded to initialize objects with a variety of initial values.
- Such constructors have the **same name**, but **different signatures** (types of parameters, number of parameters or order of parameters).
- Since constructors are just functions → It is the same concept as **function overloading**

Overloaded Constructors: Example

```cpp
class Rectangle {
private:
    int width, height;

public:
    // Default constructor (no args) → default rectangle size for this program
    Rectangle() : width(640), height(480) {}

    // Constructor with two ints
    Rectangle(int w, int h) {
        width = w;
        height = h;
    }

    // Constructor with two doubles
    Rectangle(double w, double h) {
        width = static_cast<int>(w);
        height = static_cast<int>(h);
    }

    void display() {
        cout << "Width = " << width << ", Height = " << height
             << ", Area = " << width * height << endl;
    }
};
```

```cpp
int main() {
    Rectangle r1;              // default → (640,480)
    Rectangle r3(200, 150);    // rectangle with ints
    Rectangle r4(7.5, 3.2);    // rectangle with doubles

    r1.display();
    r3.display();
    r4.display();

    return 0;
}
```

Overloaded Constructors: Example

```
class Rectangle {
private:
    int width, height;

public:
    // Default constructor (no args) → default rectangle size for this program
    Rectangle() : width(640), height(480) {}

    // Constructor with two ints
    Rectangle(int w, int h) {
        width = w;
        height = h;
    }

    // Constructor with two doubles
    Rectangle(double w, double h) {
        width = static_cast<int>(w);
        height = static_cast<int>(h);
    }

    void display() {
        cout << "Width = " << width << ", Height = " << height
             << ", Area = " << width * height << endl;
    }
};
```

```
int main() {
    Rectangle r1;              // default → (640,480)
    Rectangle r3(200, 150);    // rectangle with ints
    Rectangle r4(7.5, 3.2);    // rectangle with doubles

    r1.display();
    r3.display();
    r4.display();

    return 0;
}
```

Cout 1: Width = 640, Height = 480, Area = 307200
Cout 2: Width = 200, Height = 150, Area = 30000
Cout 3: Width = 7, Height = 3, Area = 21

Q. Can you also give constructor one parameter int size?

Copy Constructors

How to copy values from one class to another?

The Copy Constructor

- Another way to initialize an object is by using **another object of the same type**.
  For example, you can write:

```
Rectangle r1(200, 150);
Rectangle r2 = r1;   // r2 is initialized as a copy of r1
```

- Surprisingly, you don't need to write any special constructor to make this work. C++ automatically provides one for every class — it's called the **default copy constructor**..
- The copy constructor takes a reference to an existing object and uses it to **create a new object with the same member values**.

```
Rectangle r4(r1); → another way to invoke copy constructor
```

The Copy Constructor (Two ways!)

- This causes the default copy constructor for the Distance class to perform a member-by-member copy of `rect1` into `rect2`.

```
Rectangle rect2 = rect1;
```

- A different format with exactly the same effect: `rect1`is copied member-by-member into `rect3`

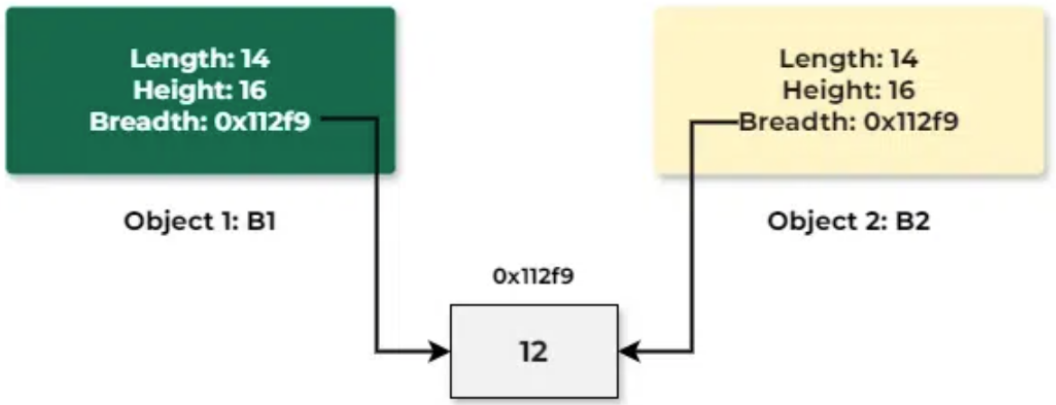
```
Rectangle rect3(rect1);
```

Copy Constructor: Shallow Vs Deep Copy

- In C++, the **default copy constructor** performs a **shallow copy**.
- A shallow copy means:
  → The values of all member variables are copied *as they are*.
  → This is fine for simple data members (like **int**, **double**, or **string**).
- But problems arise with pointers or dynamic memory:
  → If your class contains a raw pointer, the default copy constructor will simply copy the pointer's value (the memory address).
  → As a result, both the original object and the copied object will point to the **same block of memory**.
- This can cause issues like double deletion or unintended side effects.

Visual example of shallow copy

Shallow Copy Integers



Shallow Copy Pointers

Copy Constructor: How to do Deep Copy

- A **deep copy** ensures each object has **its own copy of the resource**.
- Unlike a shallow copy, it **allocates new memory** and then copies the contents from the original object.
- To implement a deep copy, we explicitly define a custom **copy constructor.** Pay attention to syntax!

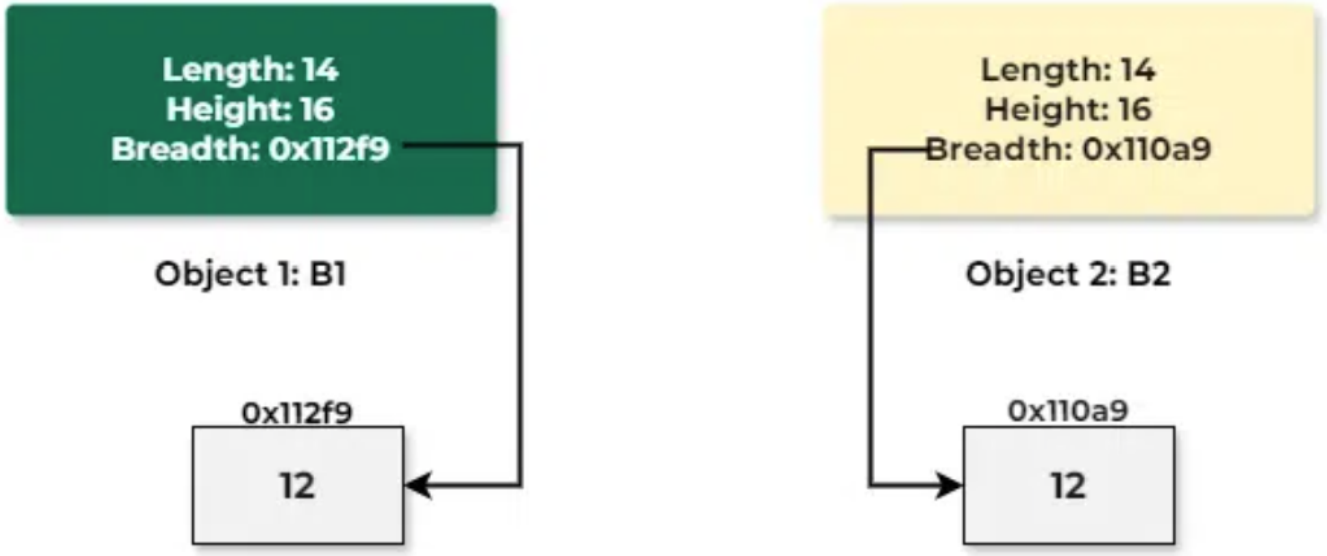
```
class Rectangle {
public:
    int* width;   // pointer to store the address of width
    int* height;  // pointer to store the address of height

    // Constructor (single line allocation + initialization)
    Rectangle(int w, int h) {
        width  = new int(w);   // allocate and initialize
        height = new int(h);   // allocate and initialize
    }

    // Deep copy constructor (step-by-step for clarity)
    Rectangle(const Rectangle& other) {
        width  = new int;          // allocate new memory and point to that new address
        *width = *other.width;     // copy the value
        height = new int;          // allocate new memory and point to that new address
        *height = *other.height;   // copy the value
    }

    // Destructor
    ~Rectangle() {
        delete width;
        delete height;
    }
};
```

Visual example of deep copy



Deep Copy Pointers

Member Functions of a Class

How to define them outside the class?

How to add class objects as arguments?

How to return class objects

Member Functions Defined Outside Class

- In C++ you can define member functions outside the class.
- To do so, you should declare a function prototype (declaration) within the class.

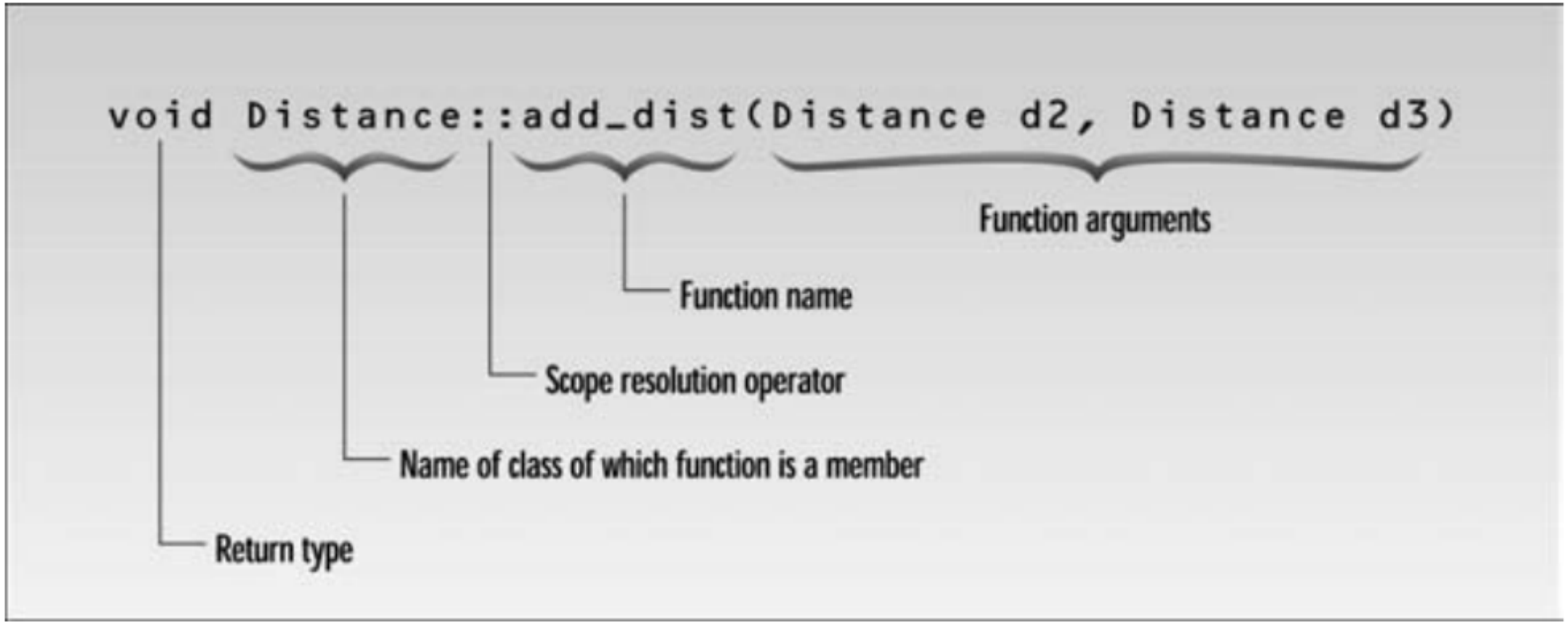
```
// function prototype (declaration)
void setDimensions(int w, int h);
```

- The function can then defined outside the class as follows:

```
// function definition outside the class
void Rectangle::setDimensions(int w, int h) {
    width = w;
    height = h;
}
```

- Observe the syntax.

Member Functions Defined Outside Class



Objects of a Class

Objects can be used as arguments of a function

also functions can return them!

Objects as Function Arguments

- **Objects** of a class can also be used as **arguments** for class member function

- The syntax for arguments that are objects is the same as that for arguments that are simple data types such as int.

- Since compareArea() is a member function of the Rectangle class, it can access the **private data members** of any Rectangle object supplied to it as

```
// Function definition outside the class
int Rectangle::compareArea(Rectangle r1, Rectangle r2) {
    int area1 = r1.width * r1.height;
    int area2 = r2.width * r2.height;

    if (area1 > area2)
        return area1;
    else
        return area2;
}
int main() {
    Rectangle rect1(4, 5);   // area = 20
    Rectangle rect2(6, 7);   // area = 42
    Rectangle rect3;        // object used to call the function

    int largerArea = rect3.compareArea(rect1, rect2);

    cout << "Area of rect1 = " << 4 * 5 << endl;
    cout << "Area of rect2 = " << 6 * 7 << endl;
    cout << "Larger Area = " << largerArea << endl;

    return 0;
}
```

Returning Objects from Functions

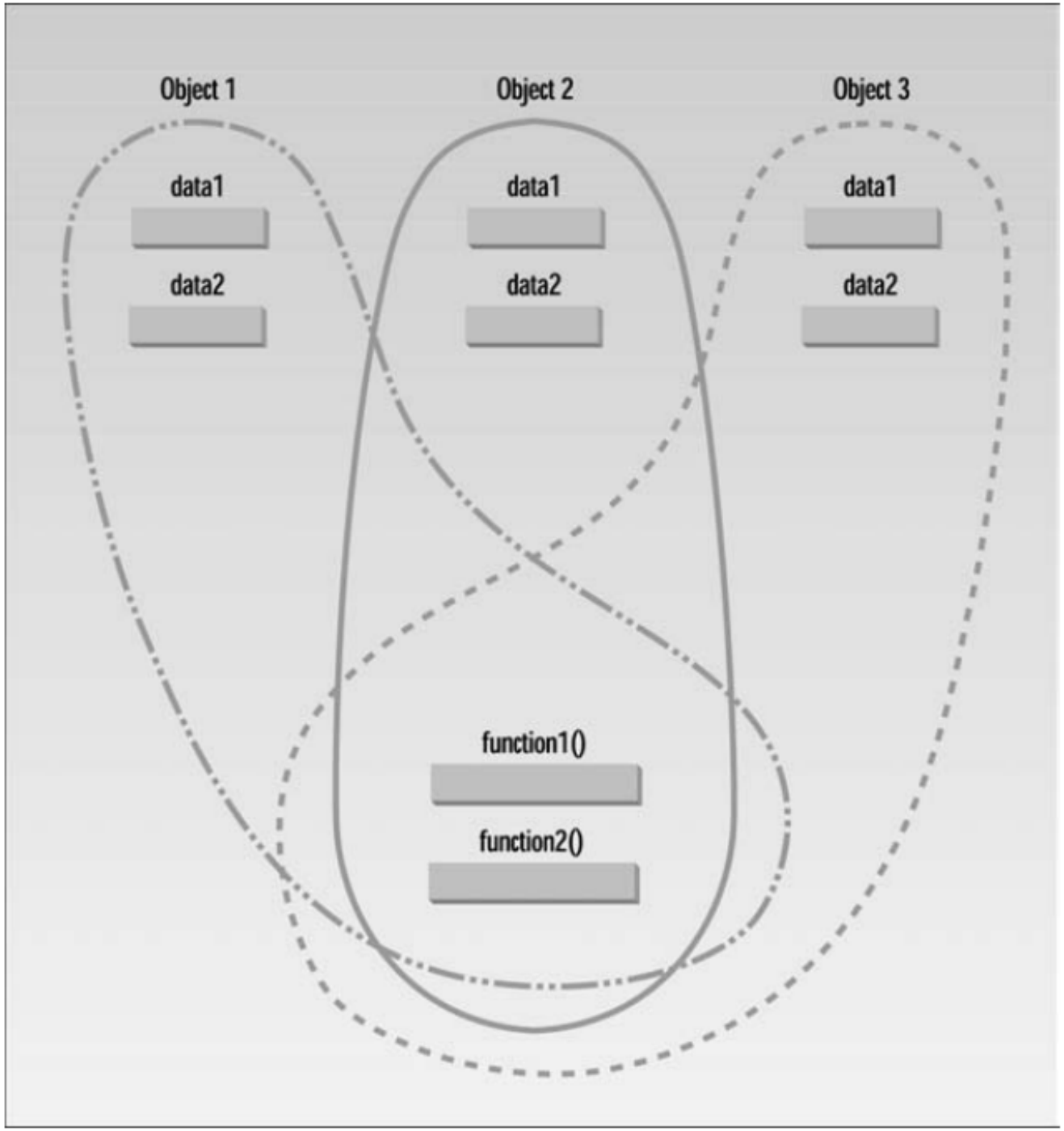
- A member function can **return an object** just like it returns an int or float.
- Here, addRect() creates a temporary Rectangle object (temp).
- The sum of the two rectangles' dimensions is stored in temp.
- The **object temp** is returned to the caller.

```
Rectangle addRect(Rectangle r2);   // 👈 declaration (prototype)
```

```
Rectangle Rectangle::addRect(Rectangle r2) {
    Rectangle temp;                // temporary object
    temp.width  = width + r2.width;
    temp.height = height + r2.height;
    return temp;                   // return by value
}
```

Classes and Objec Memory

- In terms of internal memory layout,
- All the objects in a given class use the same member functions.
- The member functions are created and placed in memory only once—when they are defined in the class definition.



Static Class Data

- One **exception** to the previous slide are **static class data**
- **Single copy** shared by **all objects**
- If a member variable is declared static, only **one copy** of it exists, regardless of how many objects are created.
- All objects of the class share the same static data member unlike normal members which are **per objects**

Destructors

How to deallocate memory in a class

Destructors

- In order to **deallocate** memory allocated to an object, a special member function called a ***destructor*** must be invoked.

- A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde.

```
class Rectangle {
private:
    int *width, *height;

public:
    // Constructor
    Rectangle(int w, int h) {
        width  = new int(w);
        height = new int(h);
        cout << "Rectangle created: "
             << *width << "x" << *height << endl;
    }

    // Destructor
    ~Rectangle() {
        delete width;
        delete height;
        cout << "Rectangle destroyed" << endl;
    }


};
```

Destructors – Some Rules

- It is **not** possible to define more than one destructor.
- Destructor **cannot be overloaded**.
- It cannot be **declared** static or const.
- Destructor **neither** requires any argument **nor** returns any value.
- It is automatically called when an object goes out of scope.
- Destructor **release memory space** occupied by the objects created by the constructor.
- In destructor, objects are destroyed **in the reverse** of an object creation.

Summary

- This session was the second look at Object Oriented Programming.
- Constructors can be **overloaded** similar to function overloading to take multiple signature types
- Values can be initialized in a class using **copy constructors**
- We have to write a **custom copy constructor** for copying pointer objects
- We can take class **objects as arguments** and also **return them** in the output
- Destructors are used to deallocate memory